

shelf (가칭)

CLI-native Research Library Agent

제품/아이디어 기획서 - 수정본

문서 목적

본 문서는 현재까지 정리한 제품 구상, 설계 방향, UX 원칙, use-case, 예시 시나리오, 기술 스택, 제약사항을 실제 구현 참고용으로 통합한 기획서다. 이번 수정본은 제품명을 shelf(가칭)로 통일하고, TUI 중심 CLI UX, Python/Rich 기반 구현 방향, Notion 비의존 local mode, slash command 기능 표를 보강했다.

제품명	shelf (가칭)
제품 유형	Local-first research library agent / topic-first source watcher / personal knowledge compiler
기본 인터페이스	CLI REPL + TUI. Slash command는 argument-heavy 명령이 아니라 command palette와 guided workflow의 진입점으로 사용한다.
선택 인터페이스	Notion sync. Notion은 GUI surface와 review workspace 역할이며 canonical store가 아니다.
Local-only 지원	Notion 없이도 Markdown directory, YAML frontmatter, SQLite, local vector index, Rich/Textual TUI로 완전 동작해야 한다.
작성일	2026-06-04

목차

1. Executive Summary
 2. 제품 정의와 핵심 포지셔닝
 3. 대상 사용자와 핵심 use-case
 4. UX 원칙 - CLI가 아니라 TUI 기반 research workspace
 5. Slash command 설계 및 사용 예시
 6. 핵심 데이터 모델과 library 구조
 7. Local-only mode와 Notion optional mode
 8. 주요 workflow와 예시 시나리오
 9. 시스템 아키텍처
 10. 구현 언어/프레임워크
 11. MCP, LLM gateway, tool layer
 12. MVP 범위와 로드맵
 13. 제약사항, 리스크, 대응 원칙
 14. 성공 지표와 구현 체크리스트
- Appendix. 예시 스키마와 로컬 디렉토리 구조

1. Executive Summary

shelf(가칭)는 사용자가 관심 있는 주제를 자연어로 입력하면 관련 source universe를 스스로 탐색하고, 신뢰도와 관련성을 평가하고, 필요한 source를 watchlist 후보로 제안하며, 시간이 지날수록 citation이 보존된 개인용 research library와 living wiki를 구축하는 CLI-native agent다.

핵심은 '북마크 관리자'가 아니다. shelf는 주제 기반 discovery, source scoring, semantic diff, digest, compilation, stale claim detection, local wiki maintenance를 결합한 개인용 research librarian agent다. 사용자는 terminal에 계속 붙어서 작업하지만, 경험은 argument-heavy CLI가 아니라 Rich/Textual 기반 TUI와 guided workflow에 가깝다.

핵심 판단

제품의 wedge는 '사용자가 등록한 사이트를 감시하는 도구'가 아니라 '사용자의 관심 주제를 이해하고, 관련 source를 스스로 발견하며, 검토 가능한 library artifact로 편찬하는 local-first agent'다. Notion은 좋은 visual surface가 될 수 있지만, Notion이 없어도 local directory 기반으로 동일한 작업이 가능해야 한다.

1.1 제품을 한 문장으로

shelf(가칭)는 CLI/TUI에서 동작하는 local-first research library agent로, 사용자의 자연어 관심 주제를 출발점으로 웹과 로컬 문서를 수집, 추적, 비교, 요약, 편찬하여 Notion 또는 로컬 디렉토리 기반 개인용 지식 라이브러리로 관리한다.

1.2 핵심 차별화

차별화 축	설명	제품적 의미
Topic-first	사이트 등록이 아니라 '이 주제가 궁금하다'에서 시작한다.	사용자가 source를 모르는 상태에서도 agent가 동작한다.
Source discovery	웹 검색, RSS, sitemap, docs, changelog, MCP source를 통해 source 후보를 찾는다.	수동 watchlist manager가 아니라 능동적 research librarian이 된다.
Semantic diff	새 글뿐 아니라 정책, 가격, 문서, API, 주장, 수치의 의미 있는 변경을 감지한다.	단순 changedetection이나 RSS reader보다 높은 신호 품질을 제공한다.
Compilation	누적 자료를 topic brief, landscape, digest, living wiki로 편찬한다.	수집량 증가가 아니라 지식 구조화를 제품 가치로 만든다.
Local-first	raw snapshot, parsed text, index, source ledger는 로컬 canonical store에 보존한다.	민감한 개인 library와 local LLM 요구에 대응한다.
TUI UX	Rich/Textual 기반 table, panel, wizard, diff, review queue를 제공한다.	비개발자도 terminal에서 선택/검토/승인 중심으로 사용할 수 있다.
Optional Notion	Notion은 dashboard, database, review queue, compilation page surface로만 사용한다.	GUI 개발 없이도 좋은 UX를 제공하되 vendor lock-in을 줄인다.

2. 제품 정의와 핵심 포지셔닝

2.1 제품명: shelf (가칭)

shelf는 개인의 research library, source shelf, knowledge shelf라는 이미지를 직관적으로 준다. 이름은 짧고 CLI command로도 자연스럽다. 다만 현재 단계에서는 가칭으로 둔다. 구현과 초기 사용자 반응에 따라 Scout, Gather, Sift 같은 대안도 유지할 수 있다.

```
$ shelf init ~/ResearchLibrary
$ shelf chat
$ shelf daemon start
$ shelf sync notion
```

2.2 피해야 할 포지셔닝

피해야 할 표현	이유	대체 표현
AI bookmark manager	저장/클리핑 제품처럼 보이고 차별화가 약하다.	Topic-first research library agent
AI RSS reader	새 글 알림과 요약에 갇힌다.	Source discovery + semantic diff + compilation
Notion AI plugin	Notion에 종속된 보조 기능처럼 보인다.	Local-first agent with optional Notion workspace
Generic second brain	범위가 너무 넓고 기존 PKM 제품과 구분이 어렵다.	CLI-native research librarian for watched topics and cited compilations

2.3 최종 포지셔닝

shelf(가칭)는 사용자가 관심 있는 주제를 자연어로 표현하면, agent가 관련 source universe를 발견하고, high-signal source를 watchlist 후보로 제안하며, 변경분과 새 자료를 검토 가능한 digest와 living wiki로 편찬하는 local-first research librarian이다.

3. 대상 사용자와 핵심 use-case

3.1 초기 ICP

사용자군	왜 필요한가	대표 산출물
독립 연구자 / 작가 / newsletter writer	여러 source를 장기간 추적하고 topic brief, 주간 digest, 글 초안을 만들어야 한다.	Weekly digest, source map, cited essay brief
창업자 / PM / 전략 담당자	시장, 경쟁사, 제품 문서, changelog, pricing page 변화를 꾸준히 봐야 한다.	Competitor update, feature landscape, stale claim report
투자자 / analyst	회사 블로그, docs, 공시, 가격 정책, hiring page, technical signal 을 추적해야 한다.	Market memo, watchlist, update alert
기술 문서/AI tool power user	LLM, MCP, agent framework, local model endpoint 등 빠르게 변하는 자료를 추적한다.	CLI agent landscape, docs diff, tool matrix
대학원생 / 연구자	논문, arXiv, 학회 자료, 블로그를 모아 literature review로 편찬한다.	Annotated bibliography, method comparison, related work draft
개인 지식관리 사용자	Notion/Obsidian/로컬 Markdown과 연결해 큰 개인 digital library를 만들고 싶다.	Local wiki, topic folders, reviewed source library

3.2 핵심 use-case

Use-case	사용자 입력	Agent 동작	결과
Topic discovery	'MCP remote server 동향이 궁금해'	쿼리 확장, 웹 검색, source 후보 수집, source scoring, one-time synthesis	초기 research note, source map, watchlist 제안
Source watch	'이 주제를 계속 추적해줘'	추천 source를 watchlist로 등록하고 주기적 check + monthly discovery refresh 구성	주간 digest, material change alert, source health
Semantic diff	'/diff' 또는 weekly run	이전 snapshot과 새 snapshot을 비교하고 의미 있는 변경만 추출	변경 요약, stale wiki claim, update patch
Compilation	'/compile' 또는 '지금까지 모은 자료로 정리해줘'	library retrieval, 중복 제거, claim extraction, 근거 검증, outline 생성	cited wiki page, market map, timeline, FAQ
Review queue	'/review'	새 item, source candidate, stale claim, failed extraction, wiki patch를 priority별로 제시	Approve/reject/snooze/mute
Local document import	'/import ~/Downloads/Papers'	PDF/HTML/Markdown parsing, card 생성, index, topic 연결	local library folder와 source ledger
Notion sync	'/sync notion'	curated metadata, review queue, compilation page를 Notion에 반영	Notion dashboard/database/page
Local-only library	Notion 없이 사용	Markdown directory, YAML, SQLite, Rich/Textual TUI로 동일 작업	디렉토리별 정리된 research library

4. UX 원칙 - CLI가 아니라 TUI 기반 research workspace

shelf는 CLI 기반 agent이지만, UX를 전통적인 Unix command의 argument 조합으로 제한하지 않는다. Codex CLI 나 Claude Code류 도구처럼 사용자가 지속적으로 session에 붙어 작업하되, slash command는 command palette 와 guided workflow의 entry point 역할을 한다.

4.1 TUI를 제품 정체성으로 둔다

Python 기반 구현에서는 Rich를 명시적으로 사용한다. Rich는 table, panel, markdown rendering, progress bar, status display, colored diff preview 같은 terminal-native UI 요소를 제공한다. 더 full-screen에 가까운 GUI-like terminal UX가 필요하면 Textual을 사용한다. 즉 기본 terminal 경험은 '명령어 암기'가 아니라 '선택, 검토, 승인' 중심이어야 한다.

UX 구성요소	구현 후보	역할
Command palette	prompt_toolkit 또는 Textual command screen	사용자가 '/'만 입력해 기능을 탐색하고 fuzzy search로 진입한다.
Tables / panels	Rich	source 후보, review queue, watcher health, job status를 읽기 쉽게 보여준다.
Progress / job monitor	Rich progress, Textual widgets	대량 fetch, parse, index, sync 작업의 진행 상황을 표시한다.
Diff viewer	Rich syntax/table + local diff generator	wiki patch, source content change, compilation update를 승인 전 확인한다.
Guided wizard	Textual form 또는 prompt_toolkit flow	/watch, /compile, /model, /notion 설정을 단계형 UI로 처리한다.
Evidence drawer	Textual side panel 또는 Rich expandable block	각 요약/판단의 원문 source, snapshot, page/section을 표시한다.

4.2 Slash command 원칙

원칙	설명
Arguments are optional	'/watch add URL --freq weekly'는 고급 사용자용 accelerator다. 기본 UX는 '/watch'를 입력하면 guided flow가 뜨는 방식이다.
Natural language is primary	사용자는 '이 주제 알고 싶어'라고 입력할 수 있고, agent가 /explore 또는 /track workflow로 라우팅한다.
Plan before mutation	source 등록, wiki update, Notion sync, 파일 작성 전 항상 계획과 diff를 제시한다.
Approve / reject / mute	library 오염을 방지하기 위해 자동 수집보다 자동 제안과 사용자 검토를 우선한다.
Status is visible	현재 workspace, model, remote tool, pending jobs, review queue 수를 항상 보여준다.
[Shelf: ~/ResearchLibrary] [model: local-qwen] [remote: off] [sources: 42] [inbox: 18] [review: 5] shelf> /review	

5. Slash command 설계 및 사용 예시

아래 명령은 최종 CLI 문법이 아니라 제품 UX 설계안이다. 핵심은 각 명령이 단순 함수 호출이 아니라 guided TUI workflow를 여는 점이다. argument는 scriptable/advanced layer로 유지한다.

5.1 탐색, 수집, 추적 명령

Command	기능	TUI 흐름	사용 예시
/	Command palette를 연다.	명령 목록, fuzzy search, 최근 작업, 추천 next action 표시	/ -> compile 검색 -> Enter
/explore	자연어 주제에서 one-time research와 source discovery를 실행한다.	주제 범위 확인 -> 검색 깊이 선택 -> source map 생성 -> watch 후보 제안	/explore 주제: local-first document agents
/track	일회성 research 주제를 지속 추적 topic으로 승격한다.	추천 source 검토 -> frequency 선택 -> discovery refresh 설정 -> 승인	/track 이 주제를 weekly로 추적
/watch	개별 source 또는 watched topic을 관리한다.	source health dashboard -> add/run/pause/mute/fix 선택	/watch 1. Add source URL 붙여넣기
/sources	source universe와 source 후보를 검토한다.	Pinned/Watched/Candidate/Muted/Rejected 탭 표시	/sources 후보 source 8개 검토
/clip	URL 또는 현재 clipboard의 article을 즉시 저장한다.	URL 분석 -> collection/topic 제안 -> summary card 생성	/clip https://example.com/post
/import	로컬 PDF/HTML/Markdown 폴더를 library로 가져온다.	폴더 선택 -> 파일 유형 요약 -> parse/index 계획 -> 승인	/import ~/Downloads/Papers

5.2 검토, 요약, 편찬 명령

Command	기능	TUI 흐름	사용 예시
/inbox	새로 수집된 item을 빠르게 훑는다.	High-signal / worth saving / noise 그룹 표시	/inbox 중요 5개만 보기
/review	source candidate, stale claim, failed extraction, wiki patch를 처리한다.	Pending queue -> evidence -> approve/reject/snooze/mute	/review Source candidates 승인
/digest	기간/collection/topic 기준 digest를 생성한다.	기간 선택 -> source 범위 선택 -> high-signal threshold -> 생성	/digest 이번 주 AI agents
/compile	누적 자료를 topic brief, landscape, FAQ, timeline 등으로 편찬한다.	산출물 유형 선택 -> 범위/기간/source 확인 -> draft -> diff -> apply	/compile 'CLI agent platform landscape'
/wiki	local wiki 또는 Notion page를 update/browse/rollback한다.	wiki tree -> stale page -> update patch -> 승인	/wiki update mcp-tooling
/diff	source snapshot, wiki page, compilation draft의 변경분을 검토한다.	before/after, semantic summary, evidence 표시	/diff openai-docs --since last-month
/search	library 전체에서 keyword/semantic search를 수행한다.	query -> filters -> ranked results -> open/save/compile	/search OpenAI-compatible endpoint
/ask	library와 웹 source에 근거한 Q&A를 수행한다.	local only/web 포함 선택 -> citation-backed answer	/ask 지난 3개월 agent CLI 변화는?

5.3 설정, 연동, 안전 명령

Command	기능	TUI 흐름	사용 예시
/model	OpenAI-compatible endpoint와 역할별 model profile을 설정한다.	provider 선택 -> base_url -> model test -> capability probe	/model Ollama local-qwen 등록
/mcp	local/remote MCP server를 등록하고 권한을 관리한다.	server add/list/inspect -> trust level -> permission prompt	/mcp add firecrawl remote read-only
/privacy	로컬/원격 처리, Notion sync, MCP 전송 범위를 점검한다.	데이터가 외부로 나가는 경로와 rule 표시	/privacy audit
/notion	Notion 연결, schema 생성, sync mode를 관리한다.	token/OAuth -> parent page -> DB 생성 -> sync mode	/notion setup
/local	Notion 없이 사용하는 local directory workspace를 관리한다.	folder tree, markdown export, index health, local wiki path 표시	/local open /local rebuild
/sync	Notion 또는 local markdown output과 동기화한다.	변경 사항 preview -> conflict 처리 -> apply	/sync notion /sync local
/jobs	background watcher, parser, indexer, sync 작업을 모니터링한다.	running/failed/scheduled jobs, retry/stop/resume	/jobs failed extraction 재 시도
/rules	분류, source scoring, digest threshold, writing style rule을 관리한다.	rule list -> add/edit/test -> 적용 범위 선택	/rules add 'SEO aggregator는 mute 후보'
/export	Markdown, JSON, ZIP, Notion backup 용 export를 만든다.	format 선택 -> scope 선택 -> output path -> 생성	/export markdown --collection ai
/rollback	최근 wiki update, sync, local write 작업을 되돌린다.	transaction list -> diff -> rollback 승인	/rollback last

5.4 기본 command palette 예시

shelf> /

Commands

/explore 관심 주제로 source discovery와 초기 research 실행
 /track research topic을 지속 추적 topic으로 승격
 /review 새 item, source 후보, stale claim, patch 검토
 /digest 기간/collection 기준 요약 생성
 /compile 누적 자료를 cited wiki/brief로 편찬
 /wiki local wiki 또는 Notion page 관리
 /model local/OpenAI-compatible model 설정
 /privacy 외부 전송 및 권한 점검

6. 핵심 데이터 모델과 library 구조

shelf의 제품 품질은 LLM 답변 자체보다 library state를 얼마나 안정적으로 관리하는지에 달려 있다. 따라서 모든 작업은 source, item, snapshot, claim, compilation, review action을 분리해 기록해야 한다.

6.1 핵심 객체

객체	의미	대표 필드
Topic	사용자가 추적하고 싶은 관심 주제	name, intent, scope, collections, discovery_policy, output_policy
Source	RSS, blog, docs, changelog, local folder, MCP source 등 추적 가능한 출처	url, role, status, relevance, authority, noise_risk, extraction_health
Item	source에서 수집된 개별 글/문서/페이지	title, url, published_at, captured_at, summary, topic_links, novelty
Snapshot	수집 당시 원본 또는 normalized content	hash, raw_path, normalized_path, fetched_at, parser_version
Claim	요약/편찬에 사용된 source-backed 주장 단위	claim_text, evidence_refs, confidence, stale_status
ReviewItem	사용자가 판단해야 하는 작업 단위	type, priority, suggested_action, status, evidence, local/notion ref
Compilation	여러 source를 종합해 만든 산출물	title, topic, source_count, claims, confidence, output_path
WatchRun	watcher 실행 결과	started_at, sources_checked, new_items, changes, failures, cost, model_used

6.2 Source status taxonomy

Status	의미	기본 처리
Pinned	사용자가 명시적으로 신뢰하고 고정한 source	항상 우선 확인, digest에서 높은 가중치
Watched	정기적으로 확인하는 source	주기적 check, diff, digest
Candidate	발견되었지만 watch 전인 source	review queue에서 approve/reject
Ephemeral	one-time research에만 사용한 source	source ledger에는 남기되 watch하지 않음
Muted	관련은 있지만 노이즈가 큰 source	검색/추천에서 낮은 가중치
Rejected	사용자가 제외한 source	동일 domain/source 재추천 제한
Failing	fetch/parse/login 등 문제가 있는 source	fix flow 또는 pause 제안

6.3 Source scoring

자동 discovery가 제품 핵심인 만큼 source scoring은 필수다. 단순 relevance만 보지 않고 authority, originality, freshness, extractability, update frequency, uniqueness, noise risk, watchability를 함께 본다.

```
source_score:
  relevance: 0.92
  authority: 0.84
  originality: 0.77
  freshness: 0.88
  update_frequency: 0.63
  extractability: 0.95
  uniqueness: 0.71
  noise_risk: 0.22
  watchability: 0.86
```

7. Local-only mode와 Notion optional mode

7.1 원칙: Notion은 optional surface, 로컬은 canonical store

Notion 연동은 매우 유용한 GUI layer이지만, 제품이 Notion에만 의존해서는 안 된다. Notion이 없는 사용자는 local directory tree, Markdown file, YAML frontmatter, SQLite index, Rich/Textual TUI로 동일한 핵심 기능을 사용할 수 있어야 한다.

설계 원칙

Local users are not second-class users. Notion에서는 표, 페이지, 인라인 데이터베이스로 보이던 것이 local mode에서는 주제별 디렉토리, Markdown index, YAML 설정 파일, TUI table로 대응되어야 한다.

7.2 Notion 기능과 local 대응 구조

Notion에서 보이는 것	Local-only 대응	비고
Topics database	Topics/<topic>/topic.yaml + README.md + sources.md	topic 설정과 설명을 사람이 읽을 수 있게 보존
Sources database	Sources/*.source.yaml + Topics/<topic>/sources.md	source score, status, health를 YAML/Markdown에 기록
Items database	Items/YYYY/MM/<slug>.md	article metadata, summary, local snapshot ref 포함
Review Queue database	Review/pending/*.md, Review/approved, Review/rejected + SQLite state	TUI에서 table로 표시하고 로컬 파일로도 확인 가능
Digests page	Digests/2026-W23-<topic>.md	주간/월간 digest 보관
Compilations pages	Compilations/*.md 또는 Topics/<topic>/compilations/*.md	living wiki 또는 고정 research memo
Inline database view	Markdown table + SQLite query + TUI table	로컬에서는 생성된 table과 TUI filter로 대응
Page status property	YAML frontmatter status + CLI/TUI action	승인/거절/보류 상태를 로컬에도 남김
Notion comments/edits	Markdown edits + optional review notes	사용자 수정 사항을 local canonical version에 merge 가능

7.3 Local directory layout

```
~/ResearchLibrary/
Dashboard.md
Inbox/
  2026-06-04_new_items.md
  review_queue.md
Topics/
  Local-first Document Agents/
    topic.yaml
    README.md
    sources.md
    candidates.md
    digests/
    compilations/
    watch_runs/
Sources/
  openai-blog.source.yaml
  mcp-docs.source.yaml
Items/
  2026/06/article_slug.md
Wiki/
  ai/local-llm-endpoints.md
  ai/mcp-tooling.md
Digests/
Compilations/
Review/
  pending/ approved/ rejected/ stale_claims/
Ledgers/
  source_ledger.jsonl
  claim_ledger.jsonl
.shelf/
  library.sqlite
  jobs.sqlite
  index/
  snapshots/
  normalized/
  cache/
  config.yaml
```

7.4 Notion sync mode

Sync mode	설명	기본 추천
Curated	topics, sources, review queue, summaries, digests, compilations만 Notion에 동기화한다. raw snapshot과 embedding은 로컬 보존.	기본값
Metadata only	title, URL, status, score, local ref만 동기화한다.	privacy 민감 사용
Full public archive	public web article 본문까지 Notion에 저장한다.	선택적, rate limit 주의
Off	Notion을 전혀 사용하지 않는다.	Local-only 사용자

8. 주요 workflow와 예시 시나리오

8.1 시나리오 A - 자연어 prompt에서 topic discovery 시작

```
shelf> 로컬 LLM이나 OpenAI-compatible endpoint를 쓰는 문서 작업 agent 시장을 알고 싶어.
```

Agent는 이 입력을 단순 Q&A가 아니라 research workflow로 해석한다. 먼저 범위를 제안하고, 웹 검색 및 source discovery를 수행한 뒤, one-time answer와 source map을 제공하고, 필요하면 topic watcher 생성을 제안한다.

단계	Agent 동작	사용자에게 보이는 것
1. Intent 해석	주제, 포함 범위, 제외 범위, 시간성 판단	'이 주제를 one-time research로 볼지, living topic으로 만들지 선택하세요'
2. Query planning	동의어/관련어/source role별 검색 query 생성	검색 깊이: Fast / Balanced / Deep
3. Source discovery	web search, RSS/sitemap, docs, changelog, GitHub release, MCP source 탐색	candidate source 목록과 scoring
4. Synthesis	중복 제거, claim extraction, citation-backed summary 생성	초기 research brief
5. Track 제안	high-confidence source만 watchlist 후보로 추천	Approve selected watchers

8.2 시나리오 B - 주간 digest와 source candidate review

```
shelf> /digest
Collection: AI Agents
Period: Last 7 days
```

주간 digest는 새 글을 모두 나열하지 않는다. high-signal item, material source change, stale wiki claim, 새 source 후보, noisy source mute 제안을 구분한다.

```
Weekly digest: AI Agents

Sources checked: 42
New items: 31
Material changes: 6
Stale wiki claims: 3
Suggested source additions: 4
Noisy sources to mute: 2

Actions:
[r] review high-signal items
[s] review source candidates
[w] update wiki pages
[n] mute noisy sources
```

8.3 시나리오 C - Notion 사용자의 review workflow

Notion을 쓰는 사용자는 Review Queue database에서 source candidate의 status를 Approved로 바꿀 수 있다. shelf는 /sync notion을 통해 이를 감지하고 로컬 canonical watchlist에 반영할 계획을 제시한다.

```
shelf> /sync notion

Notion approvals found:
- Add 3 source candidates to watchlist
- Apply 1 wiki patch
- Reject 2 low-signal sources

Apply locally? [Enter] apply [v] view changes [x] cancel
```

8.4 시나리오 D - Notion 없는 local-only 사용자

Local-only 사용자는 Notion database 대신 디렉토리 와 Markdown 파일로 같은 정보를 본다. TUI에서는 source table, review queue, diff viewer가 표시되고, 실제 산출물은 Topics, Sources, Review, Compilations, Wiki 디렉토리에 정리된다.

```
shelf> /local
```

Local library status:

```
Root: ~/ResearchLibrary
Topics: 12
Sources: 84
Inbox: 18
Pending review: 9
Wiki pages: 37
```

Actions:

1. Open folder
2. Rebuild local markdown indexes
3. Check stale pages
4. Export ZIP backup

8.5 시나리오 E - Compilation 생성

```
shelf> /compile
```

What should I create?

1. Topic brief
2. Market map
3. Timeline
4. FAQ / explainer
5. Literature review

Topic: CLI 기반 agent platform landscape

Agent는 기존 library와 새 웹 discovery 결과를 함께 사용해 outline을 만들고, source-backed claim만 본문에 넣는다. 불확실한 주장, 상충되는 source, citation이 약한 내용은 Review Queue로 보낸다.

9. 시스템 아키텍처

9.1 전체 구조

CLI/TUI REPL

- Rich/Textual UI
- slash command router
- guided workflow
- |
- v

Agent Orchestrator

- intent router
- planner/workflow engine
- approval gate
- review queue
- |
- +--> Model Gateway
 - | - OpenAI-compatible endpoints
 - | - local/remote role profiles
 - |
- +--> Tool Runtime
 - | - built-in local tools
 - | - MCP local/remote tools
 - |
- +--> Ingestion/Watcher
 - | - web search, RSS, sitemap, HTML, PDF
 - | - snapshots and semantic diff
 - |
- +--> Library Store
 - SQLite metadata
 - local filesystem snapshots
 - vector index
 - source/claim ledgers
 - Markdown/Notion sync

9.2 모듈별 책임

모듈	책임	구현상 주의점
CLI/TUI layer	slash command, status bar, command palette, review UI, diff UI	Rich/Textual 기반. argument-heavy CLI는 advanced layer로만 제공
Intent Router	자연어와 slash command를 workflow로 라우팅	'알고 싶어'를 /explore로, '계속 추적'을 /track으로 해석
Discovery Agent	web/source universe 탐색, source scoring, source map	auto-watch가 아니라 proposal 중심
Watcher	known source check와 discovery refresh를 주기적으로 실행	RSS/sitemap 우선, browser automation은 fallback
Diff Engine	text diff, structural diff, semantic diff, stale claim detection	minor wording change는 digest noise로 처리하지 않음
Compiler	topic brief, digest, wiki page, comparison table 생성	citation 없는 claim은 본문에서 제외 또는 review
Local Store	canonical metadata, snapshot, normalized text, vector index, transaction log	Notion과 독립적으로 완전 동작
Notion Adapter	curated surface sync, schema 생성, review status import	rate limit, conflict, privacy 정책 준수
MCP Registry	local/remote tool 등록 및 trust policy	remote MCP에는 private content 전송 전 승인 필요

10. 구현 언어/프레임워크

MVP는 Python-first가 가장 적합하다. 제품의 핵심이 웹앱 UI가 아니라 로컬 문서 처리, CLI/TUI agent, source watcher, local index, Notion sync, LLM gateway, MCP connector이기 때문이다.

10.1 권장 기술 스택

영역	추천	이유
Core language	Python 3.12+	문서 처리, RAG, 파일 시스템, CLI/TUI, local automation 생태계가 강하다.
CLI	Typer	명령형 endpoint와 scriptable advanced mode에 적합하다.
Terminal UI	Rich + Textual	Rich로 table/panel/progress/diff를 만들고, Textual로 full-screen TUI와 wizard를 구현한다.
REPL	prompt_toolkit 또는 Textual input	autocomplete, history, command palette, multi-line input에 유리하다.
Agent orchestration	Custom lightweight workflow engine	approval, transaction, review queue, source ledger가 핵심이라 범용 agent framework에 의존하지 않는다.
LLM client	OpenAI-compatible abstraction	Ollama, LM Studio, vLLM, LiteLLM, OpenAI-compatible gateway를 동일 interface로 지원한다.
Document parsing	Docling primary, Unstructured fallback	PDF/DOCX/PPTX/XLSX/HTML/email/image 등 다양한 문서 형식 처리.
HTML/article	htpx, feedparser, BeautifulSoup, trafilatura/readability	RSS/sitemap 우선 수집과 article body extraction.
Storage	SQLite + local filesystem	metadata, job, review state는 SQLite. snapshot과 Markdown artifact는 filesystem.
Vector index	LanceDB 또는 Chroma	local-first semantic search와 retrieval.
Scheduler	APScheduler + asyncio	개인용 local daemon과 periodic watcher에 충분하다.
Notion	Notion REST API connector	schema 관리, page rendering, review status sync를 first-class adapter로 구현한다.
MCP	MCP Python SDK 또는 stdio/http client	remote/local tools를 등록하되 permission layer를 별도로 둔다.
Secrets	keyring / OS keychain	Notion token, API key를 plaintext config에 저장하지 않는다.
Testing	pytest, pytest-httpx, vcrpy, ruff, mypy/pyright	crawler/parser/sync/approval logic regression 방지.

10.2 TypeScript는 언제 필요한가

초기 MVP에는 TypeScript가 필수는 아니다. 다만 browser extension, hosted OAuth server, optional web dashboard, public Notion OAuth flow, Node ecosystem MCP server template이 필요해지는 시점에는 TypeScript를 추가하는 것이 적절하다.

10.3 Repository 구조 초안

```
shelf-agent/
  src/shelf/
    cli/
    tui/
    agent/
      router.py
      workflows/
        explore.py watch.py review.py digest.py compile.py wiki.py
    llm/
      gateway.py openai_compatible.py capability_probe.py
    ingestion/
      rss.py sitemap.py html.py pdf.py docling_loader.py
    library/
      topics.py sources.py items.py claims.py local_export.py
    index/
      sqlite_store.py vector_store.py retrieval.py
    notion/
      client.py schema.py sync.py renderer.py conflict.py
    mcp/
      registry.py client.py permissions.py
    scheduler/
      daemon.py jobs.py
    tools/
      fs.py web_fetch.py diff.py export.py
    tests/
```

11. MCP, LLM gateway, tool layer

11.1 Model gateway

shelf는 특정 모델 vendor에 묶이지 않아야 한다. OpenAI-compatible endpoint를 기본 abstraction으로 두되, endpoint별 capability profile을 반드시 관리한다. OpenAI-compatible이라고 해서 tool calling, JSON schema, vision, embeddings가 모두 안정적이라고 가정하면 안 된다.

```
models:
  planner:
    provider: openai_compatible
    base_url: http://localhost:11434/v1
    model: qwen3:32b
    capabilities:
      tools: false
      json_schema: partial
      vision: false
      embeddings: false

  embeddings:
    provider: openai_compatible
    base_url: http://localhost:11434/v1
    model: nomic-embed-text
    capabilities:
      embeddings: true
```

11.2 Built-in tools와 MCP tools의 구분

계층	포함 기능	설계 원칙
Built-in local tools	fs, parser, OCR, index, diff, export, transaction, rollback	core reliability와 safety에 직결되므로 제품 내부에서 직접 관리
MCP local tools	Zotero, local browser, custom local crawler, local DB	workspace read/write 권한은 scope와 approval로 제한
MCP remote tools	web search, crawler, SaaS connector, enterprise knowledge source	private content 전송 전 명시 승인. 기본은 read-only

11.3 Permission model

작업	기본 권한	사용자 승인
local file read	workspace 내부 허용	초기 workspace attach 시 승인
local file write	generated artifacts 경로만 허용	diff/plan 확인 후 승인
source watch 추가	candidate로만 자동 생성	watchlist 편집 전 승인
remote web search	public query만 허용	privacy setting에 따라 최초 승인
remote LLM	기본 off 또는 public source만	private local content 전송 전 별도 승인
remote MCP	read-only 기본	tool별 trust level과 data-to-send preview
Notion sync	curated mode 기본	raw snapshot/full text sync는 opt-in

12. MVP 범위와 로드맵

12.1 MVP의 단일 핵심 flow

MVP는 '사용자가 자연어로 주제를 던지면, agent가 source discovery를 수행하고, source-backed initial brief와 watchlist 후보를 만들고, local directory 또는 Notion에 review 가능한 artifact로 저장한다'는 하나의 flow에 집중한다.

User prompt

-> Topic interpretation

-> Search strategy

-> Source discovery

-> Source scoring

-> Initial synthesis

-> Watchlist proposal

-> Local/Notion review queue

-> Optional topic watcher

12.2 Phase plan

Phase	목표	주요 구현
Phase 0	Workspace skeleton	shelf init, config.yaml, SQLite schema, local directory layout
Phase 1	Local library mode	/clip, /import, HTML/PDF parsing, local Markdown output, source ledger
Phase 2	LLM gateway	OpenAI-compatible config, local model test, summary card, capability probe
Phase 3	Topic discovery	natural prompt -> web search -> source candidate -> scoring -> initial research note
Phase 4	Watcher	RSS/sitemap watcher, snapshots, diff, review queue, weekly digest
Phase 5	TUI UX	Rich/Textual /review, /watch, /compile, command palette, approval UI
Phase 6	Notion sync	schema creation, curated sync, Review Queue status import
Phase 7	MCP	stdio/remote read-only MCP registry, permission prompts

12.3 MVP에 포함하지 않을 것

제외 항목	이유
완전 자동 browser crawling	login, CAPTCHA, dynamic layout, ToS, 유지보수 비용이 큼
모든 source 자동 watch	library pollution과 notification fatigue 유발
Notion-only product	local-first 원칙과 Notion 없는 사용자 요구에 어긋남
모바일 앱	capture convenience보다 agent core 검증이 우선
팀 협업/권한 관리	개인 local-first MVP 이후 확장
완전 자율 background crawling	초기에는 명시적 run과 digest 중심

13. 제약사항, 리스크, 대응 원칙

리스크	문제	대응
웹 수집 brittle	RSS가 없거나 JS-heavy, login, paywall, CAPTCHA가 있을 수 있다.	RSS/sitemap 우선, browser fallback 후순위, source health score와 실패 표시
Library pollution	자동 discovery가 많아지면 개인 library 품질이 떨어진다.	discover freely, watch cautiously. watchlist 편입은 기본적으로 승인 필요
Notification fatigue	알림이 많아지면 사용자가 꺼버린다.	기본은 weekly digest, immediate alert는 high-signal only
Local model variability	OpenAI-compatible endpoint마다 tool/json/vision/embedding 지원이 다르다.	capability probe, fallback plan, role별 model profile
Citation fidelity	자동 compilation이 근거 없는 주장을 포함할 수 있다.	claim ledger, source snapshot, citation requirement, low-confidence review
Notion dependency	API rate limit, schema 변화, vendor lock-in 가능성	Notion은 optional adapter, canonical data는 local store
Privacy	개인 문서, browsing source, notes가 외부로 나갈 수 있다.	remote search/LLM/MCP/Notion sync를 분리하고 data-to-send preview 제공
UX complexity	CLI, TUI, local, Notion, MCP, model 설정이 복잡해질 수 있다.	guided wizard, sane defaults, local-only first-run, progressive disclosure
Legal/ToS	무분별한 crawling은 사이트 정책과 충돌할 수 있다.	public pages, RSS/sitemap 우선, rate limit, robots/ToS respect, user-owned sources

13.1 반드시 지킬 제품 원칙

- 수집을 늘리는 것이 아니라 편찬 품질을 높인다.
- 모든 generated claim은 source snapshot 또는 local document 근거를 가져야 한다.
- watchlist는 자동으로 비대해지지 않게 한다. candidate와 review queue를 기본으로 둔다.
- Notion은 GUI surface이지 canonical archive가 아니다.
- local-only 사용자는 기능상 불이익이 없어야 한다.
- TUI는 명령어 암기가 아니라 선택, preview, diff, approval 중심이어야 한다.
- Remote tool/LLM/Notion sync는 항상 어떤 데이터가 외부로 나가는지 보여줘야 한다.

14. 성공 지표와 구현 체크리스트

14.1 성공 지표

지표	의미
Time to first useful brief	처음 자연어 주제 입력 후 source-backed brief가 만들어지는 시간
Recommended source approval rate	추천 source 후보 중 사용자가 실제 watch로 승인한 비율
Digest usefulness	주간 digest에서 사용자가 save/update/wiki action을 수행한 비율
Noise rate	mute/reject된 source 또는 item의 비율
Citation coverage	compilation claim 중 source evidence가 붙은 비율
Stale claim detection	기존 wiki 내용 중 업데이트가 필요한 claim을 탐지한 수
Local-only retention	Notion 없이 local directory/TUI만으로 계속 사용하는 사용자 비율
Watcher health	주기적 run에서 fetch/parse 성공한 source 비율

14.2 구현 체크리스트

영역	체크리스트
Workspace	shelf init, config, local directory, SQLite migration, backup/export
TUI	command palette, status bar, /review, /watch, /compile wizard, diff viewer
Ingestion	RSS, sitemap, HTML article extraction, PDF import, normalized markdown
Discovery	query expansion, source candidate extraction, source scoring, source map
Watcher	scheduled run, snapshot hash, text diff, semantic diff, failed source handling
Compiler	retrieval, dedupe, claim extraction, citation check, Markdown/Notion renderer
Local mode	Topics/Sources/Items/Review/Wiki 디렉토리 생성, markdown index rebuild
Notion	schema setup, curated sync, rate limit retry, review status import, conflict handling
LLM	OpenAI-compatible profiles, capability probe, local model presets, remote use warnings
MCP	registry, stdio/http transport, trust level, data-to-send preview
Safety	approval gates, transaction log, rollback, privacy audit, secrets storage
Testing	HTTP mocking, source parser regression, Notion sync idempotency, local export tests

Appendix. 예시 스키마와 로컬 artifact

A. topic.yaml 예시

```
name: Local-first Document Agents
status: active
intent: >
  문서 수집, 요약, 편찬, 로컬 모델 기반 agent 제품 동향 추적
collections:
  - ai-agents
  - local-llm
  - mcp
discovery_policy:
  known_source_check: weekly
  open_web_refresh: monthly
  source_candidate_review: monthly
source_policy:
  auto_save_items: true
  auto_add_watchers: false
  require_approval_for_new_domains: true
outputs:
  - weekly_digest
  - living_wiki
  - source_map
  - stale_claim_report
```

B. source_candidate.yaml 예시

```
url: https://example.com/engineering-blog
name: Example Engineering Blog
role: expert_commentary
status: candidate
discovered_from:
  topic: Local-first Document Agents
  run_id: discovery_2026_0604_001
score:
  relevance: 0.86
  authority: 0.72
  watchability: 0.81
  noise_risk: 0.35
extraction_health: good
suggested_mode: weekly_digest
recommended_action: add_to_watchlist_after_review
```

C. Review item 예시

```
type: source_candidate
priority: high
status: pending
title: Add Example Engineering Blog to AI Agents topic
why_suggested:
  - 4 recent posts matched this topic
  - RSS feed available
  - extraction succeeded on sampled pages
suggested_action: watch_weekly_digest
evidence:
  - snapshot_id: snap_2026_0604_001
  - sample_items: 4
user_actions:
  - approve
  - save_as_candidate
  - mute_domain
  - reject
```

D. Final product statement

shelf(가칭)는 CLI-native이지만 TUI-first인 research library agent다. 사용자는 자연어와 slash command로 작업하고, agent는 source discovery, watch, diff, digest, compilation을 수행한다. Notion 사용자는 dashboard와 database로 결과를 검토하고, Notion이 없는 사용자는 local directory와 Markdown wiki로 동일한 library를 유지한다.